

PRACTICAL – 01

AIM: Document Indexing and Retrieval

- Implement an inverted construction algorithm.
- Build a simple document retrieval system using the constructed index.

CODE:

```
# Define the documents

document1 = "The quick brown fox jumped over the lazy dog ."
document2 = "The lazy dog slept in the sun."

# Step 1: Tokenize the documents

# Convert each document to lowercase and split it into words
tokens1 = document1.lower().split()
tokens2 = document2.lower().split()

# Combine the tokens into a list of unique terms
terms = list(set(tokens1 + tokens2))

# Step 2: Build the inverted index

# Create an empty dictionary to store the inverted index
inverted_index = {}

# For each term, find the documents that contain it
for term in terms:
    documents = []
    if term in tokens1:
        documents.append("Document 1")
    if term in tokens2:
        documents.append("Document 2")
    inverted_index[term] = documents

# Step 3: Print the inverted index
```

for term, documents in inverted_index.items():

```
    print(term, "->", ", ".join(documents))
```

OUTPUT:

```
sun. -> Document 2
. -> Document 1
lazy -> Document 1, Document 2
quick -> Document 1
jumped -> Document 1
dog -> Document 1, Document 2
brown -> Document 1
over -> Document 1
in -> Document 2
the -> Document 1, Document 2
slept -> Document 2
fox -> Document 1
```

Practical 2

AIM: Retrieval Models

- Implement the Boolean retrieval model and process queries.
- Implement the vector space model with TF-IDF weighting and cosine similarity.

Part A: Boolean Retrieval Model

Aim: Implement the Boolean retrieval model and process queries

```
docs = {  
    1: "apple banana orange",  
    2: "apple banana",  
    3: "banana orange",  
    4: "apple"  
}  
  
index = {}  
  
for doc_id, text in docs.items():  
    for word in text.split():  
        # Add the document ID to the set for that word  
        index.setdefault(word, set()).add(doc_id)  
  
all_docs = {1, 2, 3, 4}  
  
print("AND (apple & banana):", index["apple"] & index["banana"])  
print("OR (apple | orange):", index["apple"] | index["orange"])  
print("NOT (orange):", all_docs - index["orange"])
```

Part B: Vector Space Model (TF-IDF & Cosine Similarity)

Aim: Implement the vector space model with TF-IDF weighting and cosine similarity

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = [
    "This is the first document.",
    "This document is the second document.",
    "And this is the third one.",
    "Is this the first document?"
]

query = ["This is a query document."]

vec = TfidfVectorizer()

tfidf_docs = vec.fit_transform(docs) # Learn vocabulary and transform docs
tfidf_query = vec.transform(query) # Transform query based on learned vocabulary

similarity_scores = cosine_similarity(tfidf_query, tfidf_docs)

print("Cosine Similarities against Query:")
print(similarity_scores)
```

Practical 3

AIM: Spelling Correcting in IR System

- Develop a spelling correction module using edit distance algorithms.
- Integrate the spelling correction module into an information retrieval system.

```
import difflib

docs = {
    "Doc1": "information retrieval is fun",
    "Doc2": "spelling correction module"
}

valid_words = ["information", "retrieval", "spelling", "correction", "system"]
query = "informatino"

matches = difflib.get_close_matches(query, valid_words, n=1, cutoff=0.5)

if matches:
    corrected_word = matches[0] # Take the best match
    print(f"Did you mean: '{corrected_word}'?")
    print("Search Results:")
    for doc_id, text in docs.items():
        if corrected_word in text:
            print(f" -> Found in {doc_id}")
else:
    print("No suggestions found. Search failed.")
```

PRACTICAL – 04

AIM: Evaluation Metrics for IR Systems

- Calculate precision, recall, and F-measure for a given set of retrieval results.
- Use an evaluation toolkit to measure average precision and other evaluation metrics.

Part A: Manual Calculation (Precision, Recall, F-Measure)

The Documents

```
retrieved = {"doc1", "doc2", "doc3"}
```

```
relevant = {"doc1", "doc4"}
```

Find True Positives, False Positives, False Negatives using basic set math

```
tp = len(retrieved & relevant) # Intersection (Correctly retrieved)
```

```
fp = len(retrieved - relevant) # Difference (Wrongly retrieved)
```

```
fn = len(relevant - retrieved) # Difference (Missed documents)
```

Calculate the metrics (with safety checks to avoid division by zero)

```
precision = tp / (tp + fp) if (tp + fp) else 0
```

```
recall = tp / (tp + fn) if (tp + fn) else 0
```

```
f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) else 0
```

```
print(f"Precision: {precision:.2f}")
```

```
print(f"Recall: {recall:.2f}")
```

```
print(f"F1-score: {f1:.2f}")
```

Part B: Using an Evaluation Toolkit

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
average_precision_score, precision_recall_curve

# Dummy Data: 1 means relevant, 0 means not relevant
y_true = [1, 0, 1, 0, 1]
y_prob = [0.8, 0.4, 0.9, 0.5, 0.7] # The IR system's confidence scores

y_pred = np.array(y_prob) >= 0.5

print(f"Accuracy: {accuracy_score(y_true, y_pred)}")
print(f"Precision: {precision_score(y_true, y_pred)}")
print(f"Recall: {recall_score(y_true, y_pred)}")
print(f"F1 Score: {f1_score(y_true, y_pred)}")
print(f"Avg Precision: {average_precision_score(y_true, y_prob)}")

precision_c, recall_c, _ = precision_recall_curve(y_true, y_prob)

plt.plot(recall_c, precision_c, color='blue')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.show()
```

PRACTICAL – 05

AIM: Text Categorization

- Implement a text classification algorithm (e.g., Naïve Bayes or Support Vector Machines).
- Train the classifier on a labelled dataset and evaluate its performance.

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score
```

```
X_train = ["buy cheap meds now", "hello my friend", "win a free phone", "let's grab lunch"]
```

```
y_train = ["spam", "ham", "spam", "ham"]
```

```
X_test = ["free cash transfer", "how is your day"]
```

```
y_test = ["spam", "ham"]
```

```
vectorizer = TfidfVectorizer()
```

```
X_train_tfidf = vectorizer.fit_transform(X_train)
```

```
X_test_tfidf = vectorizer.transform(X_test) # Note: only transform() for the test set!
```

```
svm_classifier = SVC(kernel='linear')
```

```
svm_classifier.fit(X_train_tfidf, y_train)
```

```
y_pred = svm_classifier.predict(X_test_tfidf)
```

```
print(f"Actual Labels: {y_test}")
```

```
print(f"Predictions: {list(y_pred)}")
```

```
print(f"Accuracy Score: {accuracy_score(y_test, y_pred)}")
```


PRACTICAL – 06

AIM: Clustering for Information Retrieval • Implement a clustering algorithm (e.g., K-means or hierarchical clustering).

- Apply the clustering algorithm to a set of documents and evaluate the clustering results.

```
# Import necessary libraries
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.metrics import silhouette_score
```

```
# 1. Distinct sample documents (makes explaining the clusters very easy!)
```

```
docs = [
```

```
    "apple banana orange fruit",
```

```
    "dog cat mouse pet",
```

```
    "mango kiwi fruit",
```

```
    "horse pig pet"
```

```
]
```

```
X = TfidfVectorizer().fit_transform(docs)
```

```
# We chain the .fit(X) right onto the end to save space and time
```

```
kmeans = KMeans(n_clusters=2, n_init=10).fit(X)
```

```
labels = kmeans.labels_
```

```
score = silhouette_score(X, labels)
```

```
print(f"Cluster Labels: {labels}")
```

```
print(f"Silhouette Score: {score:.2f}")
```

PRACTICAL – 07

AIM: Web Crawling and Indexing

- Develop a web crawler to fetch and index web pages.
- Handle challenges such as robots.txt, dynamic content, and crawling delays.

```
import requests
```

```
import time
```

```
from bs4 import BeautifulSoup
```

```
from urllib.parse import urljoin
```

```
start_url = "https://www.w3schools.com/python/"
```

```
visited = set()
```

```
queue = [start_url]
```

```
while queue and len(visited) < 5:
```

```
    url = queue.pop(0) # Get the first URL in line
```

```
    if url not in visited:
```

```
        try:
```

```
            time.sleep(1)
```

```
            response = requests.get(url, timeout=5)
```

```
            print("Crawled & Indexed:", url)
```

```
            visited.add(url)
```

```
            soup = BeautifulSoup(response.text, "html.parser")
```

```
            for tag in soup.find_all("a", href=True):
```

```
                full_link = urljoin(url, tag['href'])
```

```
                if full_link not in visited and full_link.startswith("http"):
```

```
                    queue.append(full_link)
```

```
            except Exception:
```

```
                Continue
```

PRACTICAL – 08

AIM: Link Analysis and PageRank

- Implement the PageRank algorithm to rank web pages based on link analysis.
- Apply the PageRank algorithm to a small web graph and analyze the results.

```
import networkx as nx
```

```
G = nx.DiGraph()
```

```
links = [  
    (1, 2), (1, 3), (1, 4), # Page 1 links to 2, 3, and 4  
    (2, 1),                # Page 2 links to 1  
    (3, 1), (3, 2), (3, 4), # Page 3 links to 1, 2, and 4  
    (4, 3)                 # Page 4 links to 3  
]
```

```
G.add_edges_from(links)
```

```
pagerank_scores = nx.pagerank(G, alpha=0.85)
```

```
print("--- PageRank Scores ---")
```

```
for page, score in sorted(pagerank_scores.items()):
```

```
    print(f"Page {page}: PageRank Score = {score:.4f}")
```

PRACTICAL – 09

AIM: Learning to Rank

- Implement a learning to rank algorithm (e.g., RankSVM or RankBoost).
- Train the ranking model using labelled data and evaluate its effectiveness.

CODE:

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import ndcg_score

X,y = make_classification(n_samples=1000,n_features=10,n_classes=2,random_state=42)
Xtr,Xte,ytr,yte = train_test_split(X,y,test_size=0.2,random_state=42)
sc = StandardScaler()
Xtr,Xte = sc.fit_transform(Xtr), sc.transform(Xte)
model = SVC(kernel='linear').fit(Xtr,ytr)
pred = model.decision_function(Xte)
print("NDCG Score:", ndcg_score(yte.reshape(1,-1), pred.reshape(1,-1)))
```

OUTPUT:

```
NDCG Score: 0.988842661357703
```